

LOW-LEVEL DESIGN

Aggregation Layer — Validate, Classify & Persist

A scheduled, stateless job that reads raw events in bounded batches, applies schema / billing / source rules to classify each event, writes to MongoDB and Parquet, and deletes raw only after both are durable. RAW_EVENTS stays the source of truth until then — giving zero loss with constant memory.

STATUS Proposed SCOPE Validation, classification & persistence

LOAD 2 billion events / month

1. Architecture Overview

A scheduled, **stateless** job reads raw events from RAW_EVENTS in **bounded batches**, validates and classifies each event, writes to MongoDB and Parquet, then deletes the raw objects **only after both sinks are durable**. RAW_EVENTS remains the source of truth until then — that is what makes the design zero-loss without buffering anything in memory.

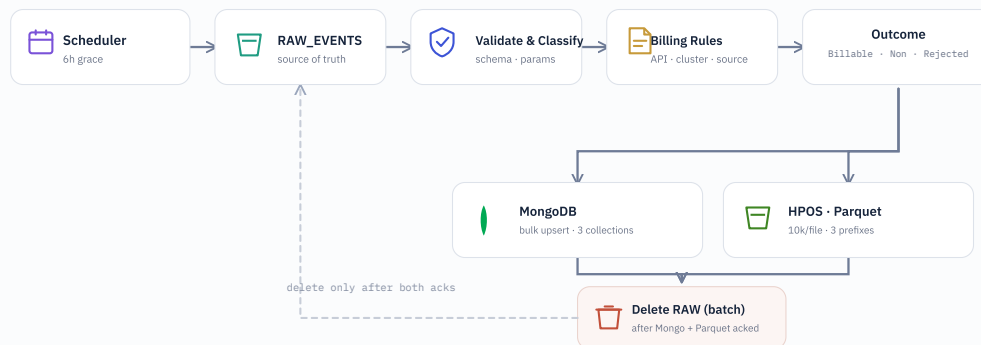


Figure 1 — Aggregation flow. Raw events are read in bounded batches, classified, written to MongoDB and Parquet, then batch-deleted from RAW_EVENTS only after both sinks acknowledge.

No data loss

Raw deleted only after Mongo upsert AND Parquet PUT are acked. Any crash leaves raw in place → safe reprocess. Idempotency by eventId → no duplicates.

No memory issues

Bucket is streamed & paginated — never loaded whole. Bounded in-flight batch (e.g. 10k) → constant memory regardless of backlog.

2. Scheduler & Grace Period

- Runs on an interval with a **6h grace window** (configurable) so late events are picked up on a later run.
- Because processed raw objects are deleted, each run only sees **unprocessed + late** events — no re-scan of finished work.
- Late files append to their event-time hour; events older than grace are flagged `late_beyond_grace` for billing true-up.

3. Reading RAW_EVENTS

- List by prefix (date/hour/partition) with **paginated batch listing**.
- Process a bounded batch (e.g. **10k events**), then release and fetch the next — constant memory.
- Per-event raw objects → use **parallel GETs across prefixes**; scale workers by prefix to sustain throughput.

4. Validation & Classification

Applied in order to each event; the first failing gate decides the outcome.

Step	Check
1 · Schema	Structural / type validation.
2 · Required params	Mandatory fields present.
3 · Source rules	API-level, cluster-level and source-specific validations.
4 · Billing rules	Decide the outcome.

Outcome per event: **Billable**, **Non-Billable**, or **Rejected**.

5. Persistence — Mongo, then Parquet

Step 1 — Write to MongoDB

- **Bulk upsert by eventId** into `billable_events` / `non_billable_events` / `rejected_events`.
- Upsert makes re-runs idempotent — no duplicate documents.
- Batched writes (not per-event) to sustain 2B/month.

Step 2 — Write Parquet to HPOS

- **10,000 events / file** (configurable) into the three outcome prefixes: `billable_events/`, `non_billable_events/`, `rejected_events/`.
- Deterministic file names (prefix + batch range) → idempotent overwrite on replay.
- Snappy compression; partitioned by event-time.

6. Delete RAW — only after both acks

The raw object is removed with a **batch delete** only once its Mongo upsert and Parquet PUT are both acknowledged. This 3-system write is not atomic, so idempotency by `eventId` covers every crash point.

Crash point	Recovery
After Mongo, before Parquet	Raw remains → reprocess; upsert dedups, Parquet written, then delete.
After Parquet, before delete	Raw remains → reprocess; upsert + deterministic Parquet overwrite, then delete.
Mongo or HPOS down	Retry with backoff; do not delete → raw stays buffered. No memory growth.

7. Concurrency Safety

- Partition the keyspace by prefix; a **lease / lock per prefix** ensures one worker owns it.
- No two workers process the same object → no double billing.
- Leases expire on worker loss → another worker safely takes over.

8. Retries & Resiliency

- Per-batch retry with **exponential backoff**; idempotency makes re-runs safe.
- Sink outage → stop deleting; raw is the durable buffer (no heap growth).
- Poison objects (repeated hard failures) → move to a quarantine/ prefix + alert. Distinct from business **Rejected**.

9. Capacity & Sizing

Proposed load is **2 billion events / month**. Memory stays constant per worker (bounded batch); scale horizontally by adding workers, each leasing distinct prefixes.

10k

Batch / parquet size

6h

Grace window

bulk

Mongo upserts

batch

S3 delete

Per-event raw input means high LIST / GET / DELETE counts — batch every API call and parallelize across prefixes to sustain throughput.